

AWS Elastic Load Balancing (ELB) - Service Overview

Introduction

Elastic Load Balancing automatically distributes incoming application traffic across multiple targets, such as EC2 instances, containers, and IP addresses, in one or more Availability Zones. ELB increases the availability and fault tolerance of your applications by ensuring traffic is not routed to unhealthy targets.

Load Balancer Types

- Application Load Balancer (ALB): operates at Layer 7 (HTTP/HTTPS), supporting content-based routing (e.g. routing based on URL path or hostname), and is the most common choice for modern web applications and microservices.
- Network Load Balancer (NLB): operates at Layer 4 (TCP/UDP/TLS), designed for extreme performance and the ability to handle millions of requests per second while preserving the client's source IP, and supports assigning a static IP or Elastic IP per Availability Zone.
- Gateway Load Balancer (GWLB): designed for deploying, scaling, and managing third-party virtual network appliances, such as firewalls and intrusion detection systems, operating transparently at Layer 3.

Target Groups

A target group routes requests to one or more registered targets (EC2 instances, IP addresses, Lambda functions, or ECS tasks) using the protocol and port number you specify, and performs health checks on those targets independently of the load balancer itself.

Health Checks

Each target group is configured with a health check, typically an HTTP request to a specified path (e.g. /health), an interval, a timeout, and healthy/unhealthy threshold counts. Targets that fail health checks are automatically removed from active rotation until they pass again, preventing traffic from reaching a broken instance or task.

Listeners

A listener checks for connection requests using the protocol and port you configure, and forwards traffic to one or more target groups based on rules you define. An ALB listener commonly listens on port 443 (HTTPS) and forwards to a target group on an internal port such as 8000.

SSL/TLS Termination

Load balancers commonly handle SSL/TLS termination, decrypting HTTPS traffic at the load balancer using a certificate managed by AWS Certificate Manager (ACM), and forwarding unencrypted (or separately encrypted) traffic to backend targets. This offloads the computational cost of encryption from application servers.

Sticky Sessions

ALBs support sticky sessions (session affinity) using an application-generated or load-balancer-generated cookie, ensuring that requests from a given client are consistently routed to the same backend target for the duration of a session, when the application requires it.

Integration with ECS and Auto Scaling

When an ECS service is configured with a load balancer, ECS automatically registers and deregisters tasks with the target group as they start and stop, and can use ALB request-count-per-target metrics to drive Service Auto Scaling decisions.

Cross-Zone Load Balancing

When cross-zone load balancing is enabled, each load balancer node distributes traffic evenly across registered targets in all enabled Availability Zones, rather than only the targets within its own zone. Without it, an uneven distribution of targets across zones (for example, three targets in AZ-a and one target in AZ-b) can result in the single target in AZ-b receiving a disproportionate share of traffic relative to the three targets in AZ-a. ALBs enable cross-zone load balancing by default at no additional charge; for NLBs it can be enabled but may incur inter-AZ data transfer charges.

Connection Draining (Deregistration Delay)

When a target is deregistered or fails a health check, the load balancer waits for a configurable deregistration delay (default 300 seconds) before completing the deregistration, during which it stops sending new requests to the target but allows in-flight requests to finish. This prevents abruptly cutting off active user requests during a deployment, scale-in event, or unhealthy target replacement.

Common Use Cases

- Distributing traffic across a fleet of EC2 instances or ECS tasks running a web application
- Path-based routing to different backend microservices from a single domain (e.g. /api to one service, /admin to another)
- Terminating TLS centrally rather than managing certificates on every individual instance

Best Practices

- Use an ALB for HTTP/HTTPS workloads unless you specifically need NLB's raw performance or static IP capabilities.
- Configure health checks against a lightweight, dependency-free endpoint (like /health) rather than a full page render, to avoid false negatives during backend slowness.
- Terminate TLS at the load balancer using ACM-managed certificates to simplify certificate rotation.